Advanced Topics in Numerics of PDEs: FEM for the Navier–Stokes Equations (SS2026)

Chapter 2: Finite Element Computations

Dr. Peter Münch[†]

[†]Institute of Mathematics, Technical University of Berlin, Germany

April 15, 2026

Table of contents

1. Motivation
2. Finite element computations
3. Software development with Linux and C/C++
4. Elliptic & parabolic problems
5. Hyperbolic problems
6. Linear solvers & multigrid
7. Stokes problem
8. Navier–Stokes problem
9. Adaptive mesh refinement
10. High-performance computing
11. Outlook & conclusions

Part 1:

Example: 1D Poisson problem

Strong form

Given Ω,

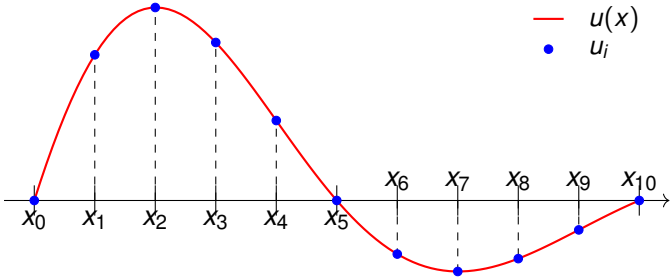
with boundary conditions

- Dirichlet boundary condition
- Neumann boundary condition

where $\partial\Omega = \Gamma_D \cup \Gamma_N$ and $\Gamma_D \cap \Gamma_N = \emptyset$.

Numerical solution approaches

Finite difference method (FDM):



Idea: Search approx. solution at points

$$u_i \approx u(x_i)$$

and approx. derivatives by FD.

Pros:

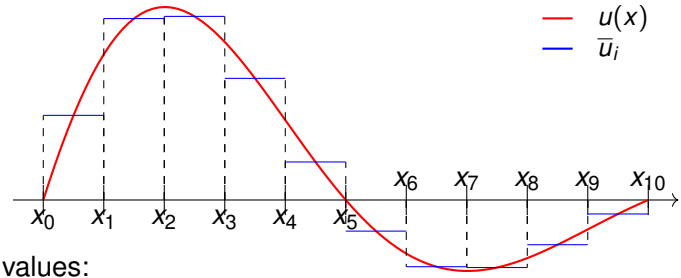
-

Cons:

-

Numerical solution approaches (cont.)

Finite volume method (FVM):



Idea: Search approx. cell-averaged values:

$$\overline{u}_i \approx \int_{x_i}^{x_{i+1}} u(x)dx \Big/ \int_{x_i}^{x_{i+1}} 1\,dx$$

Building blocks: reconstruction and fluxes at edges

Pros:

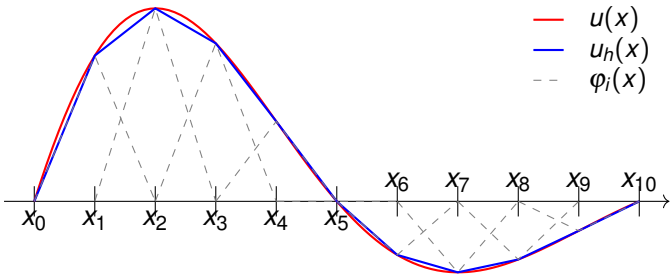
-
-

Cons:

-

Numerical solution approaches (cont.)

Galerkin framework:



Idea:

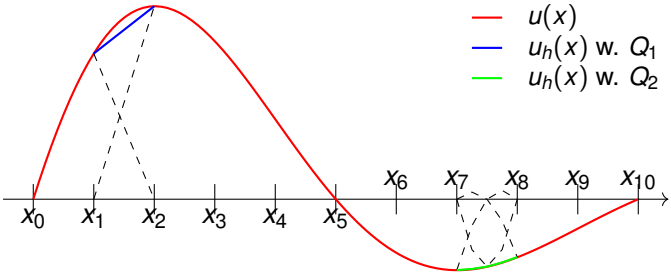
that minimizes the error $u_h - u(x)$. For this purpose, we work on the weak form.

Examples (based on the shape functions φ_i):

- spectral functions $\rightarrow$ spectral method
- hierarchical functions $\rightarrow$ hierarchical method
- piecewise functions defined on elements $\rightarrow$ finite element method
- ...

Numerical solution approaches (cont.)

Finite element method (FEM):



Piecewise functions, e.g., Lagrange function.

Pros:

-
-
-

Cons:

-
-
-

Variational formulation

The variational problem reads:

where

and the bilinear form $a(\cdot,\cdot)$ and linear functional $\ell(\cdot)$ are given by

Variational formulation: derivation

Step 1: Multiply by a test function
Let $v \in H^1(\Omega)$ with $v = 0$ on Γ_D. Multiply the equation by v and integrate over Ω:

$$\int_\Omega (-\Delta u)\, v\, dx = \int_\Omega f v\, dx.$$

Step 2: Integration by parts: Using Green's first identity:

$$\int_\Omega (-\Delta u)\, v\, dx = \int_\Omega \nabla u \cdot \nabla v\, dx - \int_{\partial\Omega} \frac{\partial u}{\partial n} v\, ds.$$

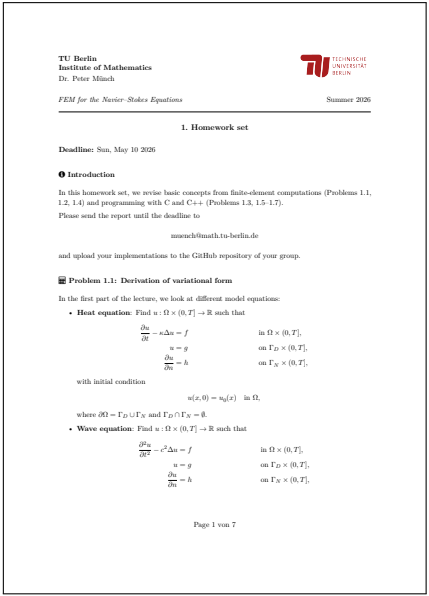
Step 3: Apply boundary conditions
Split the boundary integral. Since $v = 0$ on Γ_D, the Dirichlet term vanishes. Using the Neumann condition on Γ_N, we obtain:

$$\int_{\partial\Omega} \frac{\partial u}{\partial n} v\, ds = \int_{\Gamma_N} h v\, ds.$$

Step 4: Weak form: Thus, the weak formulation is:

$$\int_\Omega \nabla u \cdot \nabla v\, dx = \int_\Omega f v\, dx + \int_{\Gamma_N} h v\, ds.$$

Homework set 1



TU Berlin
Institute of Mathematics
Dr. Peter Münch

FEM for the Navier–Stokes Equations Summer 2026

1. Homework set

Deadline: Sun, May 10 2026

Introduction

In this homework set, we revise basic concepts from finite-element computations (Problems 1.1, 1.2, 1.4) and programming with C and C++ (Problems 1.3, 1.5–1.7).

Please send the report until the deadline to

mmunch@math.tu-berlin.de

and upload your implementations to the GitHub repository of your group.

Problem 1.1: Derivation of variational form

In the first part of the lecture, we look at different model equations:

- **Heat equation**: Find $u : \Omega \times (0,T] \to \mathbb{R}$ such that

$$\begin{aligned} \frac{\partial u}{\partial t} - \kappa \Delta u &= f && \text{in } \Omega \times (0,T], \\ u &= g && \text{on } \Gamma_D \times (0,T], \\ \frac{\partial u}{\partial n} &= h && \text{on } \Gamma_N \times (0,T], \end{aligned}$$

  with initial condition

$$u(x,0) = u_0(x) \quad \text{in } \Omega,$$

  where $\partial\Omega = \Gamma_D \cup \Gamma_N$ and $\Gamma_D \cap \Gamma_N = \emptyset$.

- **Wave equation**: Find $u : \Omega \times (0,T] \to \mathbb{R}$ such that

$$\begin{aligned} \frac{\partial^2 u}{\partial t^2} - c^2 \Delta u &= f && \text{in } \Omega \times (0,T], \\ u &= g && \text{on } \Gamma_D \times (0,T], \\ \frac{\partial u}{\partial n} &= h && \text{on } \Gamma_N \times (0,T], \end{aligned}$$

Page 1 von 7

- Task 1: Derive variational form for other model problems

Discretized weak form

Let $V_h \subset V$ and $V_{0,h} \subset V_0$ be finite-dimensional subspaces. Find $u_h \in V_h$ such that

$$a(u_h, v_h) = \ell(v_h) \quad \forall v_h \in V_{0,h}.$$

We set ($\varphi_j(x)$: shape functions, U_j: unknown coefficients)

$$u_h(x) = \sum_{j=1}^{N} U_j \varphi_j(x), \quad v_h(x) = \sum_{j=1}^{N} V_j \varphi_j(x).$$

.

Algebraic system

We get the algebraic system:

with the stiffness matrix K and load vector F given by

Algebraic system (cont.)

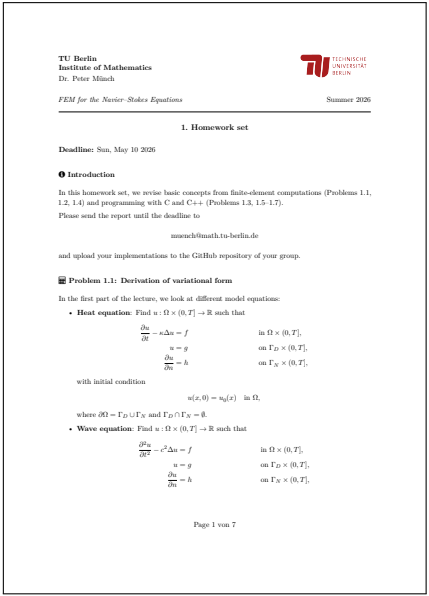
Example: 1D domain [0, 1] with homogeneous DBC at left and right end and equidistant mesh with $\Delta x = 1/(N+1)$

Stiffness matrix $K \in R^{N \times N}$:

<u>Properties:</u>

-
-
-

Homework set 1



TU Berlin
Institute of Mathematics
Dr. Peter Münch

FEM for the Navier–Stokes Equations Summer 2026

1. Homework set

Deadline: Sun, May 10 2026

Introduction

In this homework set, we revise basic concepts from finite-element computations (Problems 1.1, 1.2, 1.4) and programming with C and C++ (Problems 1.3, 1.5–1.7).

Please send the report until the deadline to

mmench@math.tu-berlin.de

and upload your implementations to the GitHub repository of your group.

Problem 1.1: Derivation of variational form

In the first part of the lecture, we look at different model equations:

- **Heat equation**: Find $u : \Omega \times (0,T] \to \mathbb{R}$ such that

$$\begin{aligned} \frac{\partial u}{\partial t} - \kappa \Delta u &= f && \text{in } \Omega \times (0,T], \\ u &= g && \text{on } \Gamma_D \times (0,T], \\ \frac{\partial u}{\partial n} &= h && \text{on } \Gamma_N \times (0,T], \end{aligned}$$

  with initial condition

$$u(x,0) = u_0(x) \quad \text{in } \Omega,$$

  where $\partial\Omega = \Gamma_D \cup \Gamma_N$ and $\Gamma_D \cap \Gamma_N = \emptyset$.

- **Wave equation**: Find $u : \Omega \times (0,T] \to \mathbb{R}$ such that

$$\begin{aligned} \frac{\partial^2 u}{\partial t^2} - c^2 \Delta u &= f && \text{in } \Omega \times (0,T], \\ u &= g && \text{on } \Gamma_D \times (0,T], \\ \frac{\partial u}{\partial n} &= h && \text{on } \Gamma_N \times (0,T], \end{aligned}$$

Page 1 von 7

- Task 2: compute mass matrix

Implementation details

In the context of FEM, we replace the integration

$$a(\varphi_i, \varphi_j) = \int_\Omega \nabla \varphi_i \cdot \nabla \varphi_j \, dx$$

by a

Implementation details (cont.)

We process all shape function that have a support on a cell together, resulting in *element matrices* K_c (partial results) that need to be *assembled* into the global stiffness matrix K.

Notation:

$$K = \sum_c R_c^\top K_c R_c,$$

where the restriction matrix R_c is mapping local indices (of shape functions) to global ones.

Part 2:

Error estimates

Interpolation error

Definition: Polynomial interpolation is given by polynomial of degree n ($p_n : [a, b] \to \mathbb{R}$):

$$p_n(x) = a_0 + a_1 x + \ldots + a_n x^n$$

such that the interpolation condition is fulfilled

$$p_n(x_i) = y_i, \quad i = 0, \ldots, n.$$

Consider that y_i is given by the function $f : [a, b] \to \mathbb{R}$:

$$y_i = f(x_i), \; i = 0, \ldots, n.$$

Definition: interpolation error at distinct data points

$$e_n(x) = f(x) - p_n(x) \quad \text{on} \quad x \in [a, b].$$

Interpolation error (cont.)

Theorem: Let $f \in C^{n+1}$ on $[a,b]$. Then, there is, for each $x \in [a,b]$, a $\xi \in [a,b]$ such that:

$$\boxed{e_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} w_{n+1}(x)} \quad \text{mit} \quad w_{n+1}(x) = (x - x_0)(x - x_1) \cdots (x - x_n).$$

This implies

$$\text{and} \quad ||\nabla e||_{L^2} \leq \frac{C(p)}{p!} h^p.$$

More details: Numerical mathematics I

A priori error estimate

Interpolation error and Galerkin orthogonality

$$(\nabla\varphi_h, \nabla(u - u_h)) = 0, \quad \forall\varphi_h \in V_h \subset H^1$$

give, for the Poisson problem, the <u>a-priori error estimates</u>:

$$\text{and} \quad ||\nabla(u - u_h)||_{L^2} < Ch^p ||\nabla^{p+1} u||_{L^2}.$$

More details: Numerical mathematics III

A posteriori error estimate

Motivation: given u_h, determine the error, e.g., in the energy norm

$$||e_h||_A^2 = A(e_h, e_h) = ?$$

Long derivation gives the <u>a posteriori error estimate</u>

$$\boxed{||e_h||_A \leq C \sqrt{\sum_T h_T^2 ||R_T||_T^2 + h_T ||J_E||_{\partial T}^2}}$$

with

- $R_K = f + \Delta u_h$
  cell residual: how well PDE is satisfied?
- $J_E = -\frac{1}{2}[[\partial_n u_h]]$
  edge residual: how smooth is the solution?

A posteriori error estimate (cont.)

Algorithm of adaptive mesh refinement (AMR):

$$\text{solve} \rightarrow \text{mark} \rightarrow \text{refine} \rightarrow \text{repeat}$$

Option: mark cells with high (a posteriori) error

$$h_T^2 ||R_T||_T^2 + h_T ||J_E||_{\partial T}^2.$$

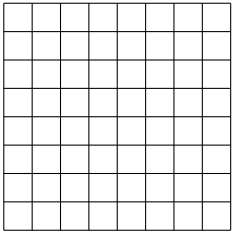
More details: Chapter 9 (AMR)

Part 3:

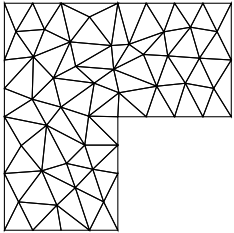
Extension to higher dimensions

Extension to 2D

structured mesh



unstructured mesh



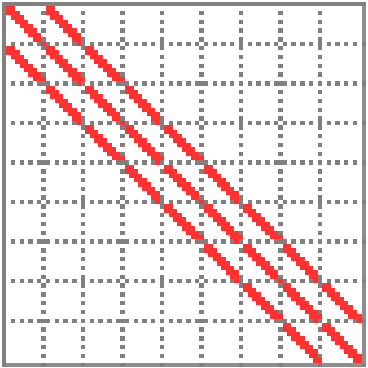
(Discrete) variational form still holds!

We only need to redefine the shape functions, e.g., linear shape functions (on triangles):

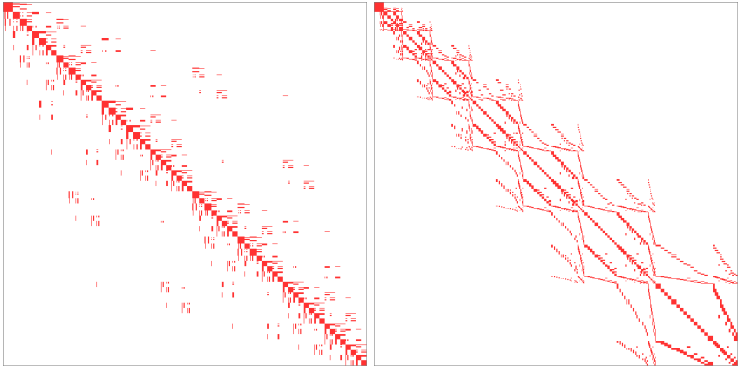
or bilinear shape functions (on quadrilaterals):

the quadrature, and the mapping (not shown; discussed later).

Sparsity patterns



structured mesh

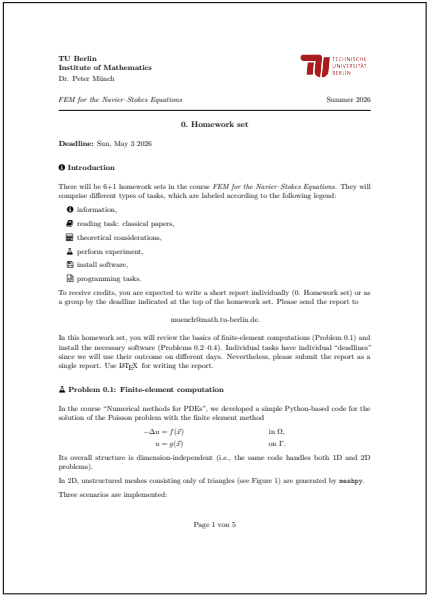


unstructured mesh

Part 4:

Implementation

Homework set 0



TU Berlin
Institute of Mathematics
Dr. Peter Münch

FEM for the Navier–Stokes Equations Summer 2026

0. Homework set

Deadline: Sun, May 3 2026

Introduction

There will be 6+1 homework sets in the course *FEM for the Navier–Stokes Equations*. They will comprise different types of tasks, which are labeled according to the following legend:

- information,
- reading task: classical papers,
- theoretical considerations,
- perform experiment,
- install software,
- programming tasks.

To receive credits, you are expected to write a short report individually (0. Homework set) or as a group by the deadline indicated at the top of the homework set. Please send the report to

muench@math.tu-berlin.de.

In this homework set, you will review the basics of finite-element computations (Problem 0.1) and install the necessary software (Problems 0.2–0.4). Individual tasks have individual "deadlines" since we will use their outcome on different days. Nevertheless, please submit the report as a single report. Use LaTeX for writing the report.

Problem 0.1: Finite-element computation

In the course "Numerical methods for PDEs", we developed a simple Python-based code for the solution of the Poisson problem with the finite element method

$$-\Delta u = f(\vec{x}) \quad \text{in } \Omega,$$
$$u = g(\vec{x}) \quad \text{on } \Gamma.$$

Its overall structure is dimension-independent (i.e., the same code handles both 1D and 2D problems).

In 2D, unstructured meshes consisting only of triangles (see Figure 1) are generated by `meshpy`.

Three scenarios are implemented:

Page 1 von 5

- Task 1: review FEM code
- Task 2-4: install required software

Implementation

In the course “Numerical methods for PDEs”, we developed a simple FEM implementation. Scope:

- unstructured meshes,
- 1D (lines) and 2D (triangles),
- linear Lagrange elements for 1D and 2D, quadratic Lagrange elements for 1D,
- Gauss quadrature,
- isoparametric mapping.

Let’s discuss the implementation!

The following slides only contain the assembly of the system matrix and the right-hand-side vector.

Implementation: assembly

⓪
①
②

```
for cell, element in zip(mesh.elements, dofs.elements):

    n_dofs_per_cell = len(element)

    # compute quadrature points, Jacobian and its determinant
    ps = [mesh.points[i] for i in cell]
    J = np.transpose(np.linalg.inv(compute_Jacobian(dim, ps)))
    detJ = compute_Jacobian_determinant(dim, ps)

    # evaluate the value and gradient of the
    # shape functions at the quadrature points
    shape_value = np.zeros([n_dofs_per_cell, n_q_points])
    shape_gradient = np.zeros([n_dofs_per_cell, n_q_points, dim])

    for q, [x_ref, w] in enumerate(quadrature):
        shape_value[:, q] = reference_shape_value(dim, fe_degree, x_ref)
        shape_gradient[:, q, :] = np.transpose(
            J @ reference_shape_gradient(dim, fe_degree, x_ref))

    # see next slide
```

Implementation: assembly (cont.)

② loop over all cells: ⓑ

```
for cell, element in zip(mesh.elements, dofs.elements):

    # see previous slide

    A_cell = np.zeros([n_dofs_per_cell, n_dofs_per_cell])
    rhs_cell = np.zeros([n_dofs_per_cell])

    for q, [x_ref, w] in enumerate(quadrature):
        value = fu_f(compute_quadrature_point(dim, ps, x_ref))

        for i in range(0, n_dofs_per_cell):
            rhs_cell[i] += shape_value[i, q] * value * detJ * w

        for i in range(0, n_dofs_per_cell):
            for j in range(0, n_dofs_per_cell):
                A_cell[i, j] += np.dot(shape_gradient[i, q, :],
                                       shape_gradient[j, q, :]) * detJ * w

    # see next slide
```

Implementation: assembly (cont.)

② loop over all cells: ⓒ

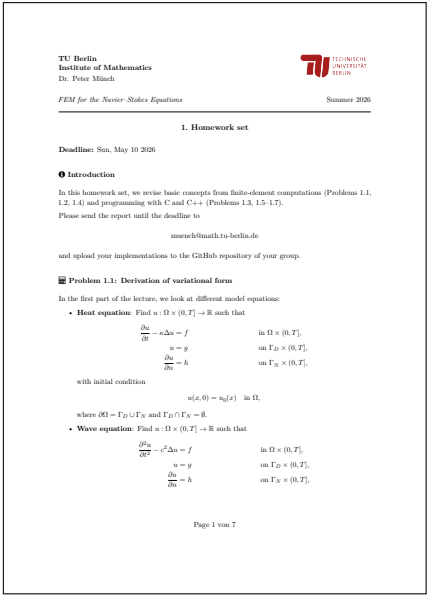
```
for cell, element in zip(mesh.elements, dofs.elements):
    # see previous slide

    # apply boundary condition
    for e, i in enumerate(element):
        if i in boundary_dofs:
            rhs_cell[e] = 0

            for j in range(0, len(element)):
                A_cell[e, j] = 0

                if not element[j] in boundary_dofs:
                    rhs_cell[j] -= A_cell[j, e] * fu_g(dofs.points[i])

                A_cell[j, e] = 0
            A_cell[e, e] = 1

    # assemble vector and matrix
    for e, i in enumerate(element):
        rhs_vector[i] += rhs_cell[e]

    for ej, j in enumerate(element):
        for ei, i in enumerate(element):
            A[i, j] += A_cell[ei, ej]
```

③

④

Workflow

Homework set 1



TU Berlin
Institute of Mathematics
Dr. Peter Münch

FEM for the Navier–Stokes Equations Summer 2026

1. Homework set

Deadline: Sun, May 10 2026

Introduction

In this homework set, we revise basic concepts from finite-element computations (Problems 1.1, 1.2, 1.4) and programming with C and C++ (Problems 1.3, 1.5–1.7).

Please send the report until the deadline to

mmench@math.tu-berlin.de

and upload your implementations to the GitHub repository of your group.

Problem 1.1: Derivation of variational form

In the first part of the lecture, we look at different model equations:

- **Heat equation**: Find $u : \Omega \times (0,T] \to \mathbb{R}$ such that

$$\begin{aligned} \frac{\partial u}{\partial t} - \kappa \Delta u &= f && \text{in } \Omega \times (0,T], \\ u &= g && \text{on } \Gamma_D \times (0,T], \\ \frac{\partial u}{\partial n} &= h && \text{on } \Gamma_N \times (0,T], \end{aligned}$$

  with initial condition

$$u(x,0) = u_0(x) \quad \text{in } \Omega,$$

  where $\partial\Omega = \Gamma_D \cup \Gamma_N$ and $\Gamma_D \cap \Gamma_N = \emptyset$.

- **Wave equation**: Find $u : \Omega \times (0,T] \to \mathbb{R}$ such that

$$\begin{aligned} \frac{\partial^2 u}{\partial t^2} - c^2 \Delta u &= f && \text{in } \Omega \times (0,T], \\ u &= g && \text{on } \Gamma_D \times (0,T], \\ \frac{\partial u}{\partial n} &= h && \text{on } \Gamma_N \times (0,T], \end{aligned}$$

Page 1 von 7

- Task 3: extend Python code with quadrilaterals

Historical background: deal.II, for a long time, only supported hypercube cells (lines, quadrilaterals, hexahedra). Support for simplex and mixed meshes was added only recently (2021).

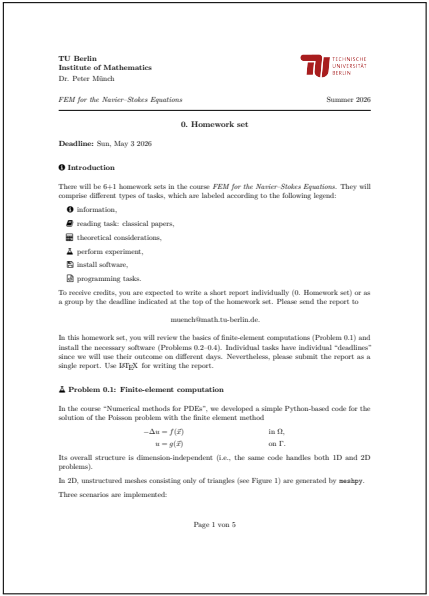
Part 5:

Conclusions & outlook

Table of contents

1. Motivation
2. Finite element computations
3. Software development with Linux and C/C++
4. Elliptic & parabolic problems
5. Hyperbolic problems
6. Linear solvers & multigrid
7. Stokes problem
8. Navier–Stokes problem
9. Adaptive mesh refinement
10. High-performance computing
11. Outlook & conclusions

Homework set 0



TU Berlin
Institute of Mathematics
Dr. Peter Münch

FEM for the Navier–Stokes Equations Summer 2026

0. Homework set

Deadline: Sun, May 3 2026

Introduction

There will be 6+1 homework sets in the course *FEM for the Navier–Stokes Equations*. They will comprise different types of tasks, which are labeled according to the following legend:

- information,
- reading task: classical papers,
- theoretical considerations,
- perform experiment,
- install software,
- programming tasks.

To receive credits, you are expected to write a short report individually (0. Homework set) or as a group by the deadline indicated at the top of the homework set. Please send the report to

muench@math.tu-berlin.de.

In this homework set, you will review the basics of finite-element computations (Problem 0.1) and install the necessary software (Problems 0.2–0.4). Individual tasks have individual "deadlines" since we will use their outcome on different days. Nevertheless, please submit the report as a single report. Use LaTeX for writing the report.

Problem 0.1: Finite-element computation

In the course "Numerical methods for PDEs", we developed a simple Python-based code for the solution of the Poisson problem with the finite element method

$$-\Delta u = f(\vec{x}) \quad \text{in } \Omega,$$
$$u = g(\vec{x}) \quad \text{on } \Gamma.$$

Its overall structure is dimension-independent (i.e., the same code handles both 1D and 2D problems).

In 2D, unstructured meshes consisting only of triangles (see Figure 1) are generated by `meshpy`.

Three scenarios are implemented:

Page 1 von 5

- Task 1: review FEM code
- Task 2: install a terminal
- Task 3: install software, e.g., gcc
- Task 4: install deal.II